CAREERTRACK

Full-Stack Job Portal and Recruitment Management System

MAJOR PROJECT REPORT

Submitted by: Shashank Shekhar

Full Stack Web Development / MERN Stack Training Program

September 2026 Batch · Project guideline-aligned implementation blueprint

Project report prepared against the supplied Minor & Major Project Guidelines. Replace or complete implementation-specific evidence after running, testing and deploying the application.

Abstract

CareerTrack is a full-stack recruitment portal designed to connect job seekers with employment opportunities and provide recruiters with a secure workspace. Public visitors can browse active jobs and submit applications without creating an account. Authorized administrators can manage job postings, review applications, update recruitment statuses, add internal notes and export application records. The system uses React for the frontend, Node.js and Express for REST APIs, and MongoDB with Mongoose for persistence.

1. Introduction and Problem Statement

Recruitment information is often distributed across multiple channels, while application tracking may rely on manual spreadsheets and email. This makes it difficult to maintain consistent job information, monitor candidate progress and securely organize résumés. CareerTrack centralizes job discovery and recruitment workflow management in one web application.

2. Objectives and Scope

Objectives: provide a responsive public job portal; enable guest applications; securely store job and application records; validate résumé uploads; provide administrator authentication and authorization; support recruitment status tracking, notes and CSV export. Scope covers one administrator role and the job/application lifecycle. Candidate accounts, automated email, interview calendar integration and production cloud file storage are future enhancements.

3. Functional Requirements

Public: browse active jobs, search by title/company/location, filter by employment type, sort, paginate, view detailed job descriptions and apply without login. Candidate form: name, email, phone, education, experience, skills, cover letter and résumé. Admin: login, dashboard statistics, job CRUD/status controls, application listing/filtering, status changes, internal notes, résumé download and CSV export.

4. Non-Functional Requirements

Responsive layout; maintainable modular code; server-side validation; password hashing; protected administrative routes; secure candidate record access; bounded upload size and type; centralized error handling; rate limiting; environment-based secrets; database indexes; and usable feedback for loading, success, empty and error states.

5. Technology Stack

Frontend: React, React Router, Axios, Vite, HTML5 and CSS3. Backend: Node.js, Express. Database: MongoDB and Mongoose. Security: bcryptjs, JWT, Helmet, CORS allowlist, express-mongo-sanitize and express-rate-limit. Uploads: Multer with PDF/DOCX allowlist and 5 MB default cap. Version control: Git/GitHub. Suggested hosting: Vercel, Render and MongoDB Atlas.

6. System Architecture and Data Flow

The React client calls Express REST endpoints over HTTP. Public job endpoints return published and unexpired jobs. Candidate submissions are multipart requests; the backend validates fields and résumé, checks the job state and relies on a unique compound MongoDB index to prevent duplicate applications. Admin requests include a bearer JWT. Authentication middleware verifies the token and loads the administrator before protected controllers execute. Mongoose persists administrators, jobs and applications.

7. Database Schema and Relationships

admins store administrator identity and bcrypt password hashes. jobs store role details, publication status, deadlines and the creating administrator reference. applications store candidate details, a job reference, résumé metadata, recruitment status and embedded internal notes. One admin creates many jobs; one job receives many applications. A unique compound index on job + normalized email enforces duplicate prevention. See docs/Database_Schema.pdf.

8. API Design

Public endpoints: GET /api/jobs, GET /api/jobs/:id, POST /api/applications/:jobId. Authentication: POST /api/auth/login. Protected endpoints: GET /api/admin/dashboard, GET/POST /api/admin/jobs, PATCH/DELETE /api/admin/jobs/:id, GET /api/admin/applications, PATCH /api/admin/applications/:id/status, POST /api/admin/applications/:id/notes, GET /api/admin/applications/:id/resume and GET /api/admin/applications/export/csv. Full method, purpose, authentication and response notes are in docs/API_Documentation.pdf.

9. Frontend Implementation

The React application uses client-side routes for the public portal, job detail, administrator login and dashboard. Reusable components include the shared layout and job card. The public portal provides keyword/location search, employment-type filter, sort controls and pagination. The detail view includes job information and a multipart application form. The administrator dashboard includes statistics, job creation and application status management. CSS includes responsive breakpoints for smaller screens.

10. Backend Implementation

The Express application separates configuration, Mongoose models, route modules, controllers, middleware and utilities. Controllers implement job discovery, job management, application submission, dashboard statistics, status changes, notes, résumé downloads and CSV generation. A centralized error middleware maps common database, validation and upload errors to HTTP responses.

11. Authentication, Authorization and Security

Administrator passwords are hashed using bcrypt. Login issues a time-limited JWT. Protected routes verify the bearer token and load the administrator record. Helmet adds security headers; CORS is restricted to configured origins; express-mongo-sanitize removes unsafe Mongo operators; rate limiting constrains API request volume. Secrets are read from environment variables. Candidate records and résumé files are exposed only through protected administrator endpoints. The demo stores JWT in sessionStorage and uses local disk for uploads; production should move to HttpOnly cookies and private object storage.

12. File Upload Handling

Multer accepts one résumé file with PDF or DOCX extension and expected MIME type. The default maximum is 5 MB, configurable with MAX_RESUME_SIZE_MB. Files receive random server-generated names. Original names are retained as metadata for authorized downloads. The upload directory must not be served as a public static directory. Production should add malware scanning and private object storage.

13. Testing Methodology and Results

A test-case workbook is included with feature, scenario, preconditions, expected result, actual result and status columns. The initial status is Not Run because runtime execution has not been performed in this generated workspace. Execute the cases against a configured MongoDB instance, record actual results and attach genuine screenshots before submission. A minimal Node test checks the health-response contract; it is not a substitute for end-to-end testing.

14. Screenshots and Output Evidence

The screenshots folder contains instructions for capturing genuine outputs from the running application. Capture the public portal, search/filter, job detail, application form and confirmation, admin login/dashboard, job management, application management, status update, authorized résumé access, CSV export, validation errors and mobile layout. Do not represent placeholder or mockup images as live implementation evidence.

15. Challenges and Solutions

Expected implementation challenges include preventing duplicate applications under concurrent submissions, protecting résumé files, handling expired jobs, keeping admin APIs private and supporting responsive tables/forms. The project addresses these with a unique compound database index, authenticated download endpoint, deadline checks, JWT middleware and responsive CSS. Record additional challenges actually encountered during implementation.

16. Limitations and Future Enhancements

Current training implementation uses local disk for uploaded files, a single administrator role and sessionStorage for the JWT. It does not include candidate accounts, email notifications, interview scheduling, audit logs, automated malware scanning or production observability. Future work can add role-based recruiter teams, secure cloud storage, email updates, interview calendars, analytics, accessibility audits and CI/CD.

17. Conclusion

CareerTrack provides the foundation for a MERN-based job portal and recruitment workflow. It demonstrates public job discovery, guest application submission, administrator authentication, job management, application tracking, MongoDB persistence, REST API design and security controls. Final project acceptance depends on completing local execution, functional testing, genuine screenshots, GitHub publication and live deployment.

18. References and Resources

React documentation: https://react.dev/ ; Node.js documentation: https://nodejs.org/docs/ ; Express documentation: https://expressjs.com/ ; MongoDB documentation: https://www.mongodb.com/docs/ ; Mongoose documentation: https://mongoosejs.com/docs/ ; Vite documentation: https://vite.dev/ ; OWASP Cheat Sheet Series: https://cheatsheetseries.owasp.org/ .